

Guide complet du workflow

Projet de stéganographie LSB et détection

Projet de traitement d'images

12 juin 2026

Table des matières

1	But du projet	2
2	Architecture logique du système	2
3	Idée d'image numérique	2
4	Algorithme d'insertion LSB	3
4.1	Structure des bits	3
4.2	Pourquoi sortir en PNG	3
5	Algorithme d'extraction	5
6	Dataset utilisé	5
7	Correction importante contre l'overfit	6
8	Visualisations dans l'application	6
8.1	Détection	7
8.2	Insertion	8
8.3	Extraction	8
9	Caractéristiques extraites	8
10	Modèles	9
11	Résultats du dernier entraînement	10
12	Structure du projet	10
13	Workflow complet d'entraînement	11
13.1	1. Préparation des paires	11
13.2	2. Split par groupe	11
13.3	3. Extraction parallèle des features	11
13.4	4. Entraînement	11
13.5	5. Test final	11
14	Trajet complet d'une image	11
15	Décisions prises pendant la réalisation	12
16	Construction des outils du projet	13

17 Ce que nous avons corrigé pendant la réalisation	14
18 Interpréter les métriques	14
19 Pourquoi les scores peuvent sembler très hauts	14
20 Contrôles et smoke tests	14
21 Détail de l'interface	15
21.1 Onglet Détecter	15
21.2 Onglet Cacher du texte	15
21.3 Onglet Extraire	15
22 Points de vigilance liés au système	15
23 Synthèse image-processing du projet	16
24 Commandes importantes	16
25 Workflow de démonstration conseillé	17
26 Limites du système réalisé	17
27 Ce qu'il faut retenir	17

1 But du projet

Ce projet montre une chaîne complète de traitement d'images autour de la stéganographie LSB. Il ne s'agit pas seulement de donner un verdict automatique. Le but est de rendre visibles les transformations : image RGB, canaux rouge/vert/bleu, plans de bits, différence amplifiée, caractéristiques statistiques, puis classification.

L'application permet trois services :

- **Détecter** si une image ressemble à une image contenant un message LSB.
- **Cacher du texte** dans une image en modifiant les bits de poids faible.
- **Extraire du texte** depuis un PNG produit par notre application.

Question centrale à savoir expliquer

La question du projet est : comment une modification presque invisible dans une image peut-elle devenir mesurable ? La réponse est que l'oeil humain regarde surtout les structures visibles, alors que le traitement d'images peut isoler les couches numériques faibles, comme les LSB, et mesurer leurs distributions.

Il faut donc présenter le projet comme une chaîne logique :

pixels -> bits -> plans LSB -> features -> modèles -> verdict

2 Architecture logique du système

Le projet est divisé en quatre blocs. Cette séparation est importante parce qu'elle permet d'expliquer clairement ce que fait chaque partie.

Bloc	Comment il fonctionne	Pourquoi il existe
Insertion	Convertit un message texte en bits et remplace les LSB des valeurs RGB.	Créer des exemples stego contrôlés et montrer concrètement la stéganographie.
Extraction	Relit les LSB, reconstruit les octets, puis décode le texte UTF-8.	Prouver que le message est vraiment dans l'image, pas seulement que l'image est classée stego.
Extraction de features	Transforme une image en nombres : LSB, textures, histogrammes et moments.	Un modèle ne comprend pas une image comme un humain ; il a besoin de mesures numériques.
Classification	Compare les features à ce qui a été appris sur clean/stego.	Donner une décision automatique et quantifiée.

Ce découpage aide aussi à comprendre et à déboguer le système : si une partie donne un mauvais résultat, on peut localiser le problème. Par exemple, si l'extraction échoue mais la détection marche, le problème peut venir du format PNG/JPEG ou de l'en-tête. Si les deux modèles se contredisent, le problème est plutôt dans la zone de décision.

3 Idée d'image numérique

Une image RGB est une matrice de pixels. Chaque pixel contient trois composantes : rouge, vert et bleu. Une valeur de composante est un entier entre 0 et 255, donc elle est représentée sur 8 bits.

Exemple :

```
Pixel RGB = (182, 91, 44)
Rouge 182 = 10110110
Vert 91 = 01011011
Bleu 44 = 00101100
```

Le bit le plus à droite est le LSB, c'est-à-dire le bit de poids faible. Le changer modifie la valeur de 1 au maximum. Par exemple, 182 peut devenir 183, ou rester 182. Cette modification est souvent invisible dans l'image affichée.

Pourquoi le LSB est discret visuellement

Dans un canal RGB, changer le LSB change une valeur de 0 ou 1. Sur une échelle de 0 à 255, c'est une variation d'environ 0,39 %. Sur une image naturelle, cette variation est noyée dans la texture, le bruit et les variations normales de couleur. C'est pour cela qu'on dit que le LSB est visuellement discret.

Mais discret ne veut pas dire indétectable. Si on modifie beaucoup de LSB pour écrire un message, on change la distribution statistique de cette couche. Le cours de traitement d'images intervient ici : on ne regarde plus seulement l'image RGB normale, on regarde des représentations transformées.

La figure suivante utilise l'image de démonstration préparée dans `presentation_assets_2`. La première ligne montre les trois composantes RGB séparées. Pour obtenir le canal rouge, on garde seulement les valeurs rouges et on met les autres composantes à zéro. On fait la même chose pour le vert et le bleu. Cette opération montre que chaque canal contient une partie différente de l'information visuelle.

La deuxième ligne montre les plans LSB. Ici, on ne garde plus l'intensité complète du canal : on garde seulement le dernier bit de chaque valeur. Un pixel noir représente un LSB égal à 0, et un pixel blanc représente un LSB égal à 1. C'est pour cela que ces images ressemblent à du bruit. Pourtant, c'est exactement cette couche qui est utilisée pour cacher le texte.

Cette figure est importante pour l'explication : elle montre que notre projet ne traite pas l'image comme un fichier abstrait. On travaille sur les matrices de pixels, on sépare les canaux, puis on isole une couche binaire. Le message caché n'est donc pas placé dans une zone visible de l'image, mais dans une représentation binaire très fine.

4 Algorithme d'insertion LSB

Notre classe `LSBSteganography` lit l'image en RGB, transforme le message en UTF-8, ajoute un marqueur de fin, puis écrit les bits dans les valeurs RGB aplaties.

4.1 Structure des bits

1. Le message est encodé en UTF-8.
2. On ajoute le marqueur «<END>».
3. On calcule la longueur du payload en bits.
4. On écrit cette longueur sur 32 bits au début de l'image.
5. On écrit ensuite les bits du payload.

Formule de capacité pour une image 512 x 512 RGB :

$$512 \times 512 \times 3 - 32 = 786400 \text{ bits utiles}$$

Cela représente environ 98 300 octets avant les limites pratiques de texte.

4.2 Pourquoi sortir en PNG

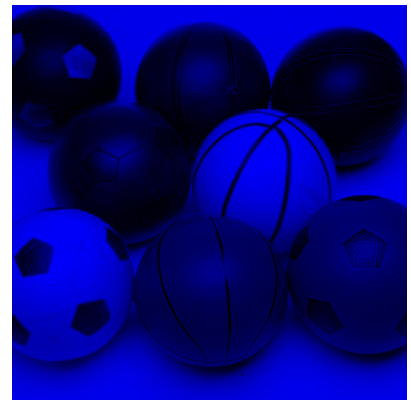
L'insertion LSB doit être sauvegardée dans un format sans perte. Si on sauvegarde en JPEG, la compression peut modifier les pixels et détruire les bits cachés. Pour cette raison, l'application produit une image PNG après insertion.



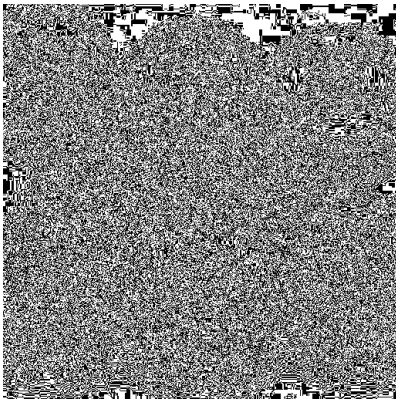
Canal rouge



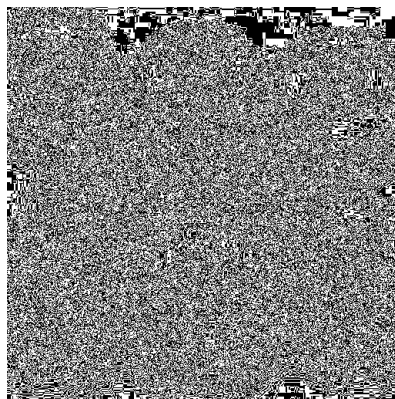
Canal vert



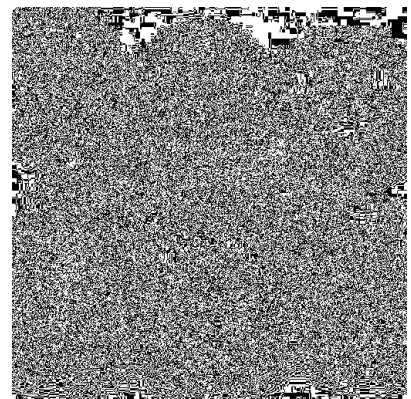
Canal bleu



Plan LSB rouge



Plan LSB vert



Plan LSB bleu

FIGURE 1 – Décomposition réelle de l'image utilisée pour la démonstration : canaux RGB puis plans LSB. Les canaux conservent une structure visuelle, alors que les plans LSB montrent seulement les bits faibles.

Pourquoi utiliser un en-tête de 32 bits

Sans en-tête, l'extracteur ne saurait pas combien de bits lire. Il pourrait lire toute l'image et produire du bruit. L'en-tête de 32 bits donne la longueur exacte du payload. C'est simple, robuste pour notre application, et facile à expliquer.

Le choix de 32 bits est volontaire : il permet de coder une longueur très grande, largement supérieure à la capacité d'une image 512 x 512 RGB. On aurait pu utiliser 16 bits pour de petits messages, mais 32 bits évite une limite artificielle et rend l'algorithme plus général.

Pourquoi ajouter un marqueur de fin

Le marqueur «<END>> sert de deuxième contrôle. L'en-tête dit combien de bits lire, et le marqueur confirme que le texte extrait se termine bien comme prévu. Si une image aléatoire a par hasard un en-tête plausible, le marqueur réduit le risque d'afficher un faux message.

Pourquoi mesurer le PSNR

Le PSNR mesure la différence entre l'image propre et l'image stego. Plus il est haut, plus la différence est faible. Dans ce projet, le PSNR sert à relier l'insertion LSB à une mesure classique de traitement d'images : nous montrons que la modification est faible non seulement visuellement, mais aussi numériquement.

La figure suivante montre le résultat de l'insertion d'un message long de 20 000 caractères. C'est volontairement plus parlant qu'un message très court : la différence amplifiée devient visible et permet de comprendre où l'algorithme a modifié les valeurs RGB. Malgré cela, l'image stego reste très proche de l'image propre dans l'affichage normal.



FIGURE 2 – Comparaison de l'image propre, de l'image contenant 20 000 caractères cachés et de la différence amplifiée. Dans cet exemple, 80 180 valeurs RGB changent, réparties sur 46 755 pixels.

La différence amplifiée ne représente pas une image naturelle. Elle sert uniquement d'outil pédagogique : on multiplie visuellement les petites différences pour les rendre observables. Sans amplification, une variation de 1 sur une valeur RGB entre 0 et 255 est généralement trop faible pour être vue directement.

5 Algorithme d'extraction

L'extraction fait l'opération inverse :

1. Lire tous les LSB des valeurs RGB.
2. Lire les 32 premiers bits comme longueur du payload.
3. Vérifier que la longueur est positive et inférieure à la capacité.
4. Lire exactement ce nombre de bits.
5. Regrouper les bits par octets.
6. Décoder en UTF-8.
7. Chercher le marqueur «<END>».

Si l'image n'a pas été produite par notre application, l'en-tête peut être invalide. L'extraction retourne alors une chaîne vide plutôt qu'un faux message.

Différence entre extraction et détection

Il faut bien distinguer ces deux notions :

- **Extraire** veut dire retrouver le texte caché. Cela fonctionne seulement si l'image respecte notre format : en-tête de 32 bits, payload UTF-8, marqueur de fin et pas de recompression destructrice.
- **Détecter** veut dire estimer si l'image ressemble statistiquement à une image stego. Cela peut fonctionner même si on ne peut pas lire le message.

Une image peut être détectée comme stego sans texte extrait. Cela arrive parce que la détection cherche des traces statistiques, alors que l'extraction cherche un protocole exact.

6 Dataset utilisé

Le dossier utilisé est `ALASKA_v2_JPG_512_QF95_COLOR`. Il contient des images JPEG couleur 512 x 512 avec qualité 95. Dans notre projet, ce dossier sert de distribution d'images propres.

Le dernier entraînement a utilisé :

- **20 000** images cover ALASKA.

- **20 000** images stego générées par notre insertion LSB.
- **28 000** images pour l'entraînement.
- **6 000** images pour la validation.
- **6 000** images pour le test.

Pourquoi utiliser ALASKA

ALASKA est un bon choix pour un projet de stéganalyse parce qu'il contient des images naturelles variées. Une image naturelle a des textures, des couleurs et des détails différents. Cela force le modèle à apprendre des règles plus générales qu'avec quelques images artificielles.

Notre dossier ne contient que les images cover préparées. Donc nous avons généré nous-mêmes la classe stego avec notre algorithme LSB. C'est une limite, mais aussi un avantage pédagogique : on contrôle exactement la méthode d'insertion et on peut montrer l'algorithme complet.

Pourquoi 512 x 512

Toutes les images sont ramenées à 512 x 512 pour avoir une taille fixe. Les features comme les histogrammes, LBP et GLCM doivent être comparables entre images. Si les tailles étaient très différentes, certaines mesures changeraient à cause de la taille plutôt qu'à cause de la stéganographie.

7 Correction importante contre l'overfit

Un ancien risque était le raccourci de format. Si les images propres restent en JPEG et les images stego sont en PNG, le modèle peut apprendre « JPEG = propre » et « PNG = stego ». Ce serait un mauvais modèle, car il ne détecterait pas vraiment la stéganographie.

La correction est la suivante :

- Chaque image ALASKA est convertie en PNG propre.
- Une copie PNG stego est générée avec le même prétraitement.
- Les deux classes ont donc le même format et la même taille.
- Le split est fait par groupe : une paire cover/stego reste dans le même ensemble.

Pourquoi cette correction est importante

Pour vérifier que le modèle n'a pas appris un raccourci, nous avons supprimé deux problèmes classiques :

1. le raccourci de format, en mettant clean et stego en PNG ;
2. le raccourci de contenu, en gardant les paires cover/stego dans le même split.

Cette partie est très importante parce qu'elle montre qu'on ne se contente pas d'obtenir une accuracy haute. On vérifie que l'accuracy correspond à la bonne raison.

8 Visualisations dans l'application

L'application ne montre pas seulement une réponse. Elle affiche des étapes visuelles réelles calculées sur l'image courante.

Comment lire les visualisations

Chaque visualisation correspond à une opération précise de traitement d'images :

- l'image RGB montre l'entrée telle qu'elle est perçue normalement ;
- les canaux rouge, vert et bleu montrent la séparation des composantes ;
- les plans LSB montrent la couche binaire où le texte est caché ;
- la différence amplifiée montre les positions où les valeurs ont changé ;
- la carte LBP résume les motifs locaux de texture ;

- les histogrammes montrent la distribution des intensités ;
- les métriques SVM/RF montrent comment les features deviennent une décision.

Ces éléments doivent être lus dans l'ordre. On commence par l'image visible, puis on descend vers les représentations numériques moins visibles, puis on termine par les features et la classification.

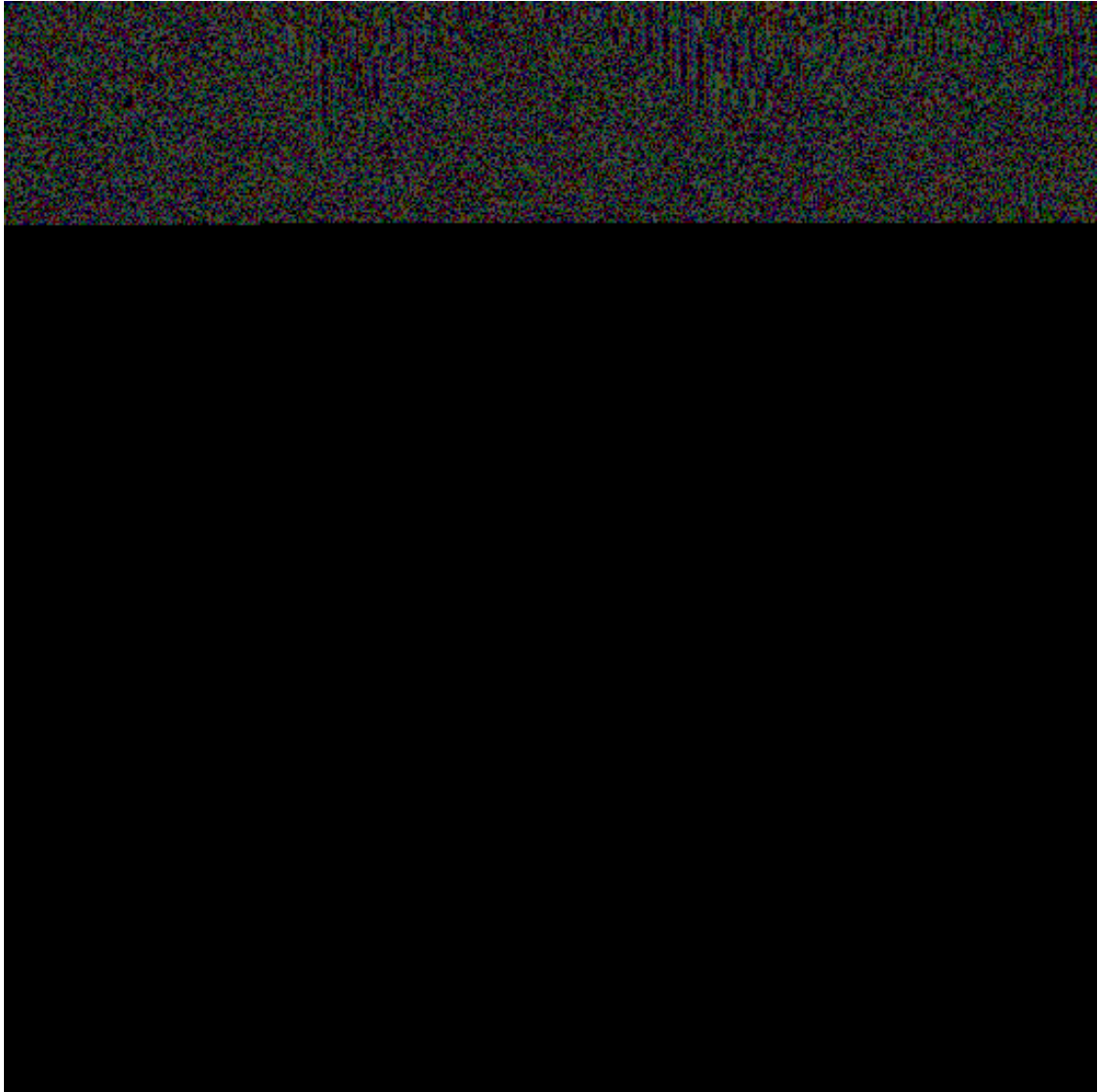


FIGURE 3 – Différence amplifiée après insertion LSB. Les zones claires indiquent les endroits où au moins une valeur RGB a changé. Cette image aide à relier l'algorithme binaire à une observation spatiale dans l'image.

8.1 Détection

Pour une image testée, l'onglet détection affiche :

- l'image RGB chargée ;
- l'image redimensionnée en 512 x 512 ;
- les trois canaux rouge, vert et bleu ;
- les histogrammes RGB ;
- les plans LSB des trois canaux ;
- une visualisation LBP ;
- les probabilités SVM et Random Forest avec leurs seuils.

8.2 Insertion

L'onglet d'insertion affiche :

- l'image propre préparée ;
- l'image stego produite ;
- la différence amplifiée ;
- la taille du message, l'en-tête de 32 bits et l'occupation de capacité ;
- le nombre de valeurs RGB modifiées ;
- le PSNR ;
- les plans LSB après insertion.

8.3 Extraction

L'onglet extraction affiche :

- les plans LSB lus ;
- la capacité disponible ;
- la longueur annoncée par l'en-tête ;
- la validité de l'en-tête ;
- le temps d'extraction ;
- le texte extrait.

9 Caractéristiques extraites

Chaque image devient un vecteur de 218 caractéristiques. Les familles principales sont :

Famille	Rôle dans le projet
En-tête LSB	Mesure si les 32 premiers bits ressemblent à une longueur valide.
Statistiques LSB	Ratios de 1, variance, test pair/impair et autocorrélation des plans LSB.
LBP	Décrit les motifs locaux de texture.
GLCM	Mesure contraste, corrélation, énergie et homogénéité des niveaux de gris.
Histogrammes RGB	Décrit la distribution des intensités dans les canaux.
Moments statistiques	Moyenne, écart type, skewness et kurtosis de chaque canal.

Pourquoi ne pas donner les pixels directement au modèle

On aurait pu donner tous les pixels à un réseau de neurones, mais ce serait plus lourd, moins explicable et moins adapté à un projet de classe centré sur le traitement d'images. Ici, les features sont choisies pour être interprétables :

- les LSB sont directement liés à la méthode de cache ;
- LBP explique les motifs locaux ;
- GLCM explique les relations spatiales entre niveaux de gris ;
- les histogrammes expliquent la distribution des couleurs ;
- les moments résument la forme statistique de chaque canal.

Le rôle des figures précédentes est donc double. Elles montrent d'abord ce que voit un humain : l'image propre et l'image stego sont proches. Ensuite, elles montrent ce que mesure le programme : plans de bits, changements de pixels et textures. Le modèle ne reçoit pas les images affichées dans le guide ; il reçoit les valeurs numériques extraites à partir de ces représentations.

Lien entre les images du guide et les 218 features

Les images du guide servent à comprendre d'où viennent les features :

- les plans LSB produisent des ratios de 0 et de 1, une variance, une autocorrélation et des mesures pair/impair ;
- les canaux RGB produisent des histogrammes et des moments statistiques ;
- la version en niveaux de gris et le canal vert servent aux mesures GLCM ;
- les voisinages locaux des pixels servent à calculer LBP.

Autrement dit, chaque figure correspond à une étape concrète du calcul. Le guide ne montre pas des illustrations décoratives : il montre les mêmes familles d'information que celles utilisées par le détecteur.

Comment expliquer LBP simplement

LBP compare un pixel avec ses voisins. Pour chaque voisin, on note s'il est plus grand ou plus petit que le pixel central. Cela produit un petit code binaire qui décrit la texture locale. Ensuite, on fait un histogramme de ces codes. C'est utile parce que la stéganographie peut modifier de petites structures locales.

Comment expliquer GLCM simplement

GLCM compte la fréquence des couples de niveaux de gris voisins. Par exemple, elle mesure combien de fois une intensité 20 est proche d'une intensité 21, ou 20 proche de 80. À partir de cette matrice, on calcule contraste, corrélation, énergie et homogénéité. C'est une méthode classique pour décrire les textures.

10 Modèles

Deux modèles sont utilisés :

- **SVM linéaire calibré** : il cherche une frontière de séparation dans l'espace des features. La calibration transforme la sortie en probabilité utilisable.
- **Random Forest calibré** : il combine plusieurs arbres de décision. Il peut mieux exploiter des interactions non linéaires entre features.

Les seuils ne sont pas choisis au hasard. Ils sont calibrés sur l'ensemble de validation pour maximiser le F1, puis testés sur un ensemble séparé.

Pourquoi deux modèles

Le SVM et le Random Forest ne raisonnent pas de la même manière. Le SVM cherche une frontière de séparation. Le Random Forest combine plusieurs arbres de décision et peut capturer des interactions plus complexes.

Afficher les deux permet de voir l'accord ou le désaccord. Si les deux donnent une probabilité haute, le verdict est plus convaincant. S'ils ne sont pas d'accord, l'application le montre, ce qui est plus honnête qu'un seul verdict caché.

Pourquoi calibrer les probabilités

Une sortie brute de modèle n'est pas toujours une vraie probabilité. La calibration rend les scores plus lisibles pour l'utilisateur. Cela ne rend pas le modèle parfait, mais cela permet d'afficher « P(stego) » de façon plus cohérente.

Pourquoi utiliser un seuil

Un modèle donne un score continu. Pour obtenir une classe propre/stego, il faut choisir un seuil. Le seuil 0,5 n'est pas toujours optimal. C'est pourquoi on l'ajuste sur la validation. Dans nos résultats, le seuil Random Forest est environ 0,7535 : il doit être plus prudent avant de dire stego.

11 Résultats du dernier entraînement

Modèle	Accuracy	Précision	Recall	F1	AUC	Seuil
SVM	0.9983	0.9967	1.0000	0.9983	0.9995	0.5000
Random Forest	0.9983	0.9980	0.9987	0.9983	0.9999	0.7535

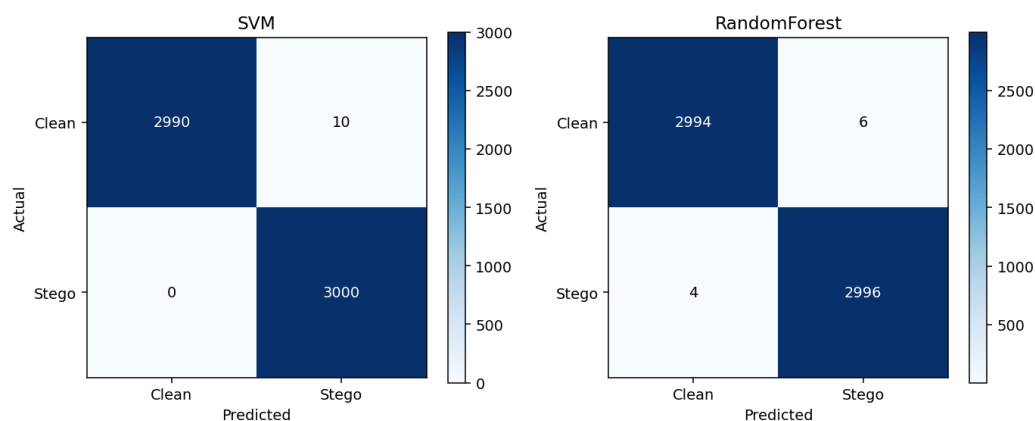


FIGURE 4 – Matrices de confusion du test.

Interprétation : les métriques sont très hautes car la tâche est bien définie : détecter notre insertion LSB sur des images ALASKA préparées. Ce n'est pas une preuve que le modèle détecte toutes les méthodes de stéganographie modernes.

12 Structure du projet

Les fichiers importants sont les suivants :

Chemin	Rôle
<code>app.py</code>	Interface Streamlit en français : détection, insertion, extraction, visualisations et métriques.
<code>detect.py</code>	Entrée ligne de commande pour tester une image ou une vidéo avec un modèle.
<code>run_pipeline.py</code>	Script principal d'entraînement : préparation, features, modèles, seuils et résultats.
<code>src/lsb_steganography.py</code>	Algorithmes d'insertion, extraction et calcul du PSNR.
<code>src/generate_dataset.py</code>	Conversion ALASKA vers PNG propre et génération des images stego LSB.
<code>src/feature_extractor.py</code>	Extraction des 218 caractéristiques.
<code>src/dataset_pipeline.py</code>	Split par groupe et extraction parallèle des features.
<code>src/classifier.py</code>	Entraînement SVM/RF, calibration, évaluation, sauvegarde.
<code>models/</code>	Modèles entraînés : SVM, Random Forest et seuils.
<code>results/</code>	Métriques JSON et figures de validation.
<code>data/processed/</code>	Splits, labels et matrices de features.

13 Workflow complet d'entraînement

13.1 1. Préparation des paires

Le script lit les images JPEG ALASKA. Pour chaque image, il crée une version propre PNG et une version stego PNG. Les deux images ont la même taille, le même format et la même origine.

Le message caché pendant l'entraînement est volontairement variable. Il peut être court, moyen ou long. Cela évite un modèle qui reconnaît seulement un seul type de payload.

13.2 2. Split par groupe

Le groupe est le nom de l'image source. La version propre et la version stego de la même image restent dans le même split. Cela évite une fuite de données : si le cover était dans train et le stego dans test, le modèle pourrait apprendre l'image au lieu d'apprendre la stéganographie.

13.3 3. Extraction parallèle des features

L'extraction des features est la partie lente. Le projet utilise `joblib.Parallel` avec plusieurs workers. Sur une machine correcte, il faut utiliser plusieurs processus, par exemple 15 workers, parce que traiter 40 000 images une par une en Python serait trop lent.

13.4 4. Entraînement

Le SVM et le Random Forest sont entraînés sur le split train. Quand l'option `-tune` est utilisée, le script cherche de meilleurs hyperparamètres. Après entraînement, les seuils sont ajustés sur la validation, pas sur le test.

13.5 5. Test final

Le test est utilisé une seule fois pour mesurer la performance finale. Les matrices de confusion et les courbes ROC sont sauvegardées dans `results/`.

14 Trajet complet d'une image

Cette section explique le projet comme une histoire complète, depuis l'image chargée jusqu'au verdict ou au texte extrait.

Image de démonstration utilisée

Dans les figures du guide, l'image propre est `presentation_assets_2/test_clean.png`. L'image stego correspondante est `presentation_assets_2/test_stego_20000.png`. Elle contient un texte extrait de 20 000 caractères. Cette paire est utile pour l'explication parce qu'elle montre clairement le principe sans sortir du fonctionnement réel de l'application.

Les valeurs mesurées pour cette paire sont :

- message extrait : 20 000 caractères ;
- valeurs RGB modifiées : 80 180 ;
- pixels touchés : 46 755 ;
- taille de l'image : 512 x 512 pixels, donc 262 144 pixels.

Ces nombres montrent que l'insertion ne modifie pas toute l'image. Elle modifie seulement les valeurs nécessaires pour aligner les LSB avec les bits du message.

Cas 1 : l'utilisateur cache un texte

1. L'utilisateur charge une image.
2. L'application convertit l'image en RGB et la redimensionne en 512 x 512.

3. Le message est converti en octets UTF-8.
4. Les octets sont convertis en bits.
5. L'application construit un en-tête de 32 bits qui contient la taille du payload.
6. Les bits sont écrits dans les LSB des valeurs RGB aplaties.
7. L'image est sauvegardée en PNG pour protéger les LSB.
8. L'application affiche l'image propre, l'image stego, la différence amplifiée, les plans LSB et le PSNR.

Pourquoi cet ordre ? Parce qu'il sépare la préparation de l'image, la préparation du message, l'insertion binaire et la validation visuelle.

Cas 2 : l'utilisateur détecte une image

1. L'image est chargée en RGB.
2. Elle est redimensionnée en 512 x 512 pour correspondre à l'entraînement.
3. Les canaux RGB sont séparés.
4. Les plans LSB sont extraits.
5. Les features LSB, LBP, GLCM, histogrammes et moments sont calculées.
6. Le vecteur de 218 valeurs est envoyé au SVM et au Random Forest.
7. Chaque modèle renvoie une probabilité.
8. La probabilité est comparée au seuil validé.
9. L'application affiche le verdict et les mesures qui l'ont produit.

Pourquoi afficher les étapes ? Parce que dans un cours de traitement d'images, le verdict seul n'est pas suffisant. Il faut montrer les transformations qui mènent au verdict.

Cas 3 : l'utilisateur extrait un texte

1. L'image PNG est chargée.
2. Les LSB de toutes les valeurs RGB sont lus dans l'ordre.
3. Les 32 premiers bits donnent la longueur du payload.
4. Si la longueur est impossible, l'application arrête l'extraction.
5. Sinon, elle lit le payload, reconstruit les octets et décode en UTF-8.
6. Elle cherche le marqueur de fin.
7. Elle affiche le texte seulement si le format est cohérent.

Pourquoi cette prudence ? Parce qu'une image quelconque contient toujours des LSB. Sans validation, on pourrait afficher du texte faux.

15 Décisions prises pendant la réalisation

Ce que nous avons fait	Pourquoi nous l'avons fait	Ce que cela évite
Nous sauvegardons les sorties en PNG	Le PNG est sans perte et conserve les LSB.	La recompression JPEG qui peut détruire le message caché.
Nous avons choisi des features interprétables	Elles correspondent à des notions de traitement d'images vues en cours.	Une boîte noire qui donne un verdict sans expliquer les transformations.

Nous utilisons SVM et Random Forest	Les deux modèles donnent deux lectures différentes des mêmes features.	Dépendre d'un seul modèle sans comparaison.
Nous avons fait un split par groupe	Une image source et sa version stego restent dans le même ensemble.	Une fuite de données qui gonfle artificiellement les scores.
Nous validons les seuils	Le score continu devient un verdict avec un seuil mesuré sur validation.	Utiliser 0,5 par habitude même quand ce n'est pas optimal.
Nous affichons les étapes visuelles	L'utilisateur voit les canaux, les plans LSB et les mesures réelles.	Une application qui cache le traitement et montre seulement une conclusion.

16 Construction des outils du projet

Outil 1 : insertion LSB

Nous avons construit l'outil d'insertion pour transformer un texte en modification mesurable de l'image. Le texte est d'abord encodé en UTF-8, puis transformé en bits. Nous ajoutons un en-tête de 32 bits pour enregistrer la longueur du message, puis un marqueur de fin pour vérifier que l'extraction est cohérente.

Ensuite, nous parcourons les valeurs RGB comme un long tableau. Pour chaque bit du message, nous remplaçons le LSB de la valeur RGB correspondante. Si le bit était déjà correct, la valeur ne change pas. Sinon, elle change seulement de 1. C'est pour cela que l'image reste presque identique visuellement.

Outil 2 : extraction LSB

Nous avons construit l'extracteur comme l'inverse exact de l'insertion. Il lit les LSB dans le même ordre. Les 32 premiers bits donnent la taille à lire. Après cela, l'outil lit le payload, reconstruit les octets et décode le texte en UTF-8.

Nous avons ajouté des validations parce qu'une image quelconque possède toujours des LSB. Sans vérifier la taille et le marqueur de fin, l'application pourrait afficher du texte faux. Donc l'outil n'affiche le message que si le format correspond à notre protocole.

Outil 3 : visualisation de traitement d'images

Nous avons ajouté des visualisations pour que l'application montre le traitement, pas seulement le résultat. Pour chaque image, elle affiche les canaux rouge, vert et bleu, les plans LSB, les histogrammes, la différence amplifiée et une carte LBP.

Ces images intermédiaires sont calculées à partir de l'image courante. Elles ne sont pas un guide statique. Elles montrent vraiment comment l'image chargée est transformée en données exploitables.

Outil 4 : extraction des caractéristiques

Nous avons construit un extracteur de features pour résumer l'image en 218 valeurs. Cela permet de passer d'une image 512 x 512 x 3, donc 786 432 valeurs RGB, à un vecteur compact. Ce vecteur garde les informations utiles pour notre problème : structure des LSB, texture, distribution des couleurs et moments statistiques.

Cette étape est le coeur image-processing du projet. Les modèles ne regardent pas directement les pixels bruts ; ils utilisent les mesures que nous avons calculées.

Outil 5 : entraînement et détection

Nous avons construit le pipeline d'entraînement pour générer les paires clean/stego, extraire les features en parallèle, entraîner les modèles, calibrer les seuils, puis sauvegarder les résultats. L'application charge ensuite les modèles sauvegardés et applique exactement le même extracteur de features sur l'image de l'utilisateur.

Cette cohérence est importante : l'image testée doit être traitée comme les images d'entraînement. Sinon, les probabilités affichées ne seraient pas fiables.

17 Ce que nous avons corrigé pendant la réalisation

Au début, le risque principal était d'obtenir de bons scores pour une mauvaise raison. Nous avons donc modifié le pipeline pour que les images propres et stego soient toutes les deux en PNG. Cela empêche le modèle d'apprendre simplement le format du fichier.

Nous avons aussi changé le split des données. Les paires venant de la même image source restent ensemble. Cela évite que le modèle voie une version d'une image pendant l'entraînement et l'autre pendant le test.

Enfin, nous avons amélioré l'application pour qu'elle affiche des étapes réelles : pas seulement des phrases comme « l'image est redimensionnée », mais les images intermédiaires, les ratios, les tailles, les plans de bits et les probabilités calculées.

18 Interpréter les métriques

La matrice de confusion a quatre cases :

- vrai propre : image propre correctement classée propre ;
- faux positif : image propre classée stego ;
- faux négatif : image stego classée propre ;
- vrai stego : image stego correctement classée stego.

Pour le SVM, la matrice est $\begin{bmatrix} 2990 & 10 \\ 0 & 3000 \end{bmatrix}$. Cela veut dire 10 faux positifs et 0 faux négatifs sur le test. Pour le Random Forest, la matrice est $\begin{bmatrix} 2994 & 6 \\ 4 & 2996 \end{bmatrix}$. Cela veut dire 6 faux positifs et 4 faux négatifs.

Le recall mesure la capacité à trouver les images stego. La précision mesure la fiabilité quand le modèle dit « stego ». Le F1 combine les deux. L'AUC mesure la qualité du classement quand on fait varier le seuil.

19 Pourquoi les scores peuvent sembler très hauts

Des scores proches de 99,8 % peuvent sembler suspects. Ils sont acceptables ici seulement si on explique la tâche exacte :

- les images stego sont produites par notre propre méthode LSB ;
- le format propre/stego est contrôlé et identique ;
- le modèle voit des indices très directs, comme l'en-tête LSB de 32 bits ;
- le test est séparé par groupe, donc il n'y a pas de paire train/test issue de la même image.

Ces scores seraient anormaux si on prétendait détecter toutes les stéganographies JPEG. Mais pour notre problème contrôlé, ils sont cohérents.

20 Contrôles et smoke tests

Avant une démonstration, il faut vérifier :

1. que `models/svm_model.pkl`, `models/rf_model.pkl` et `models/thresholds.json` existent ;
2. que `results/metrics.json` correspond au dernier entraînement ;
3. qu'une image propre ALASKA obtient une probabilité basse ;

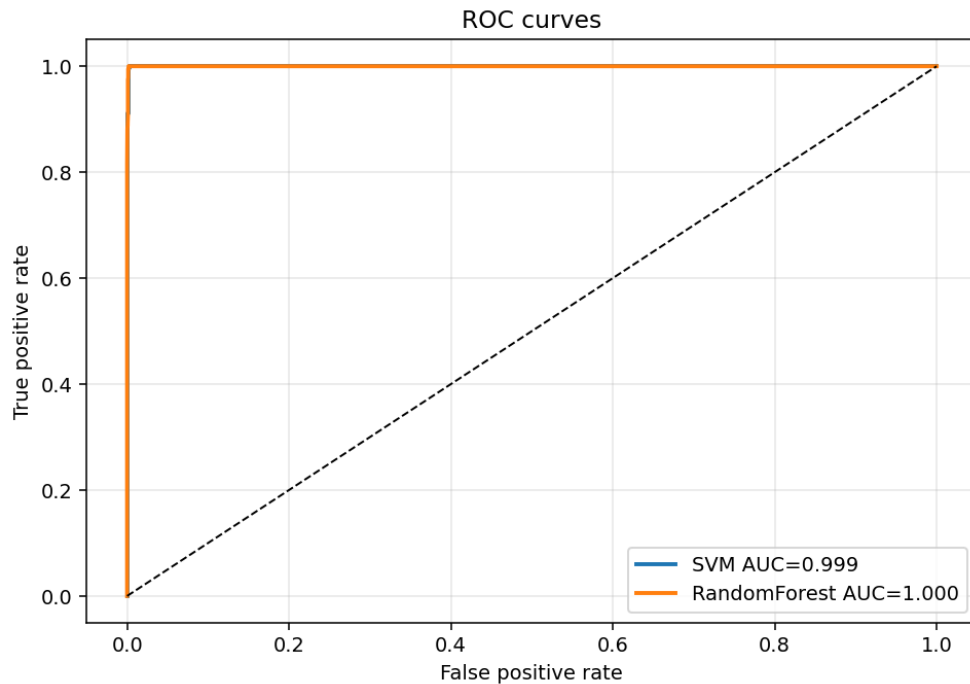


FIGURE 5 – Courbes ROC sauvegardées après l’entraînement.

4. qu’une image produite par l’onglet insertion obtient une probabilité haute ;
5. que l’extraction retrouve exactement le message.

Un test minimal consiste à cacher un court texte dans une image, lancer la détection sur l’image propre puis sur l’image stego, et extraire le message. Si ces trois opérations fonctionnent, le coeur du projet est prêt.

21 Détail de l’interface

21.1 Onglet Détecter

L’utilisateur charge une image. L’application affiche le verdict, puis les étapes visuelles sous l’image et sous le verdict. Les données affichées sont recalculées pour l’image courante : ratios LSB, en-tête, features, probabilités et seuils.

21.2 Onglet Cacher du texte

L’utilisateur peut choisir de cacher un texte ou de ne rien cacher. L’option sans texte sert de contrôle : elle produit une image comparable pour tester si le modèle invente un verdict stego. Quand un texte est caché, l’application montre la différence amplifiée, le PSNR et les plans LSB après insertion.

21.3 Onglet Extraire

L’utilisateur charge un PNG. L’application lit l’en-tête et le payload. Elle affiche les plans LSB lus, la longueur annoncée, la validité et le texte extrait.

22 Points de vigilance liés au système

Situation	Explication liée au projet
-----------	----------------------------

Le message ne s'extrait pas	L'extraction exige une image PNG non recompressée produite par notre protocole : en-tête de 32 bits, payload UTF-8 et marqueur de fin. Une capture, un JPEG ou une image modifiée peut casser les LSB.
Le détecteur classe mal une image	Le verdict dépend des features extraites et des seuils validés. Il faut lire séparément les probabilités SVM et Random Forest, surtout si l'image est proche de la frontière de décision.
Une image propre semble suspecte	Les plans LSB d'une image naturelle peuvent déjà ressembler à du bruit. Le modèle ne regarde donc pas seulement un plan LSB visuel, mais plusieurs statistiques : ratios, autocorrélation, LBP, GLCM, histogrammes et moments.
Un message très court est utilisé	Un petit payload modifie peu de valeurs RGB. La détection peut alors être moins évidente, sauf si l'en-tête LSB de notre protocole reste détectable.
Un message très long est utilisé	Plus le payload est long, plus le nombre de valeurs RGB modifiées augmente. La différence amplifiée devient plus visible, mais l'image normale peut rester proche de l'originale.

23 Synthèse image-processing du projet

La partie importante du projet est le traitement d'images :

- on manipule directement les pixels et les bits ;
- on visualise les canaux et les plans LSB ;
- on extrait des caractéristiques classiques de texture ;
- les modèles ne remplacent pas l'explication visuelle, ils utilisent les mesures produites par le traitement d'images.

Cette synthèse montre ce que nous avons vraiment construit : un système qui part des pixels, extrait des représentations visuelles et statistiques, puis utilise ces mesures pour cacher, lire ou détecter un message.

24 Commandes importantes

Installation :

```
python -m pip install -r requirements.txt
```

Entraînement complet sur les 20 000 images cover :

```
python run_pipeline.py --workers 15 --tune --force-stego --force-features
```

Entraînement rapide de test :

```
python run_pipeline.py --smoke --workers 8
```

Détection en ligne de commande :

```
python detect.py --image image.png --model rf --json
python detect.py --image image.png --model svm --json
```

Application :

```
streamlit run app.py
```

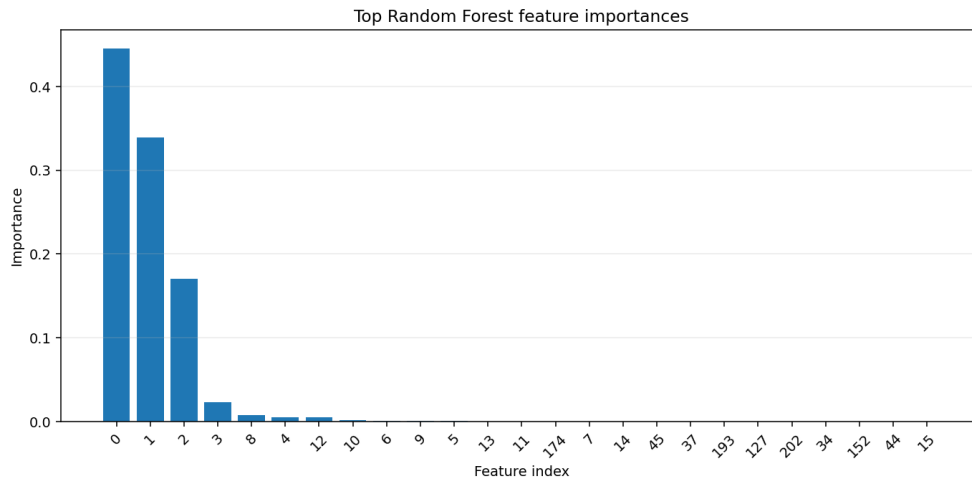


FIGURE 6 – Importance des features pour le Random Forest.

25 Workflow de démonstration conseillé

1. Ouvrir l'application Streamlit.
2. Aller dans **Cacher du texte**.
3. Charger une image, saisir un petit message, puis créer le PNG stego.
4. Montrer l'image propre, l'image stego, la différence amplifiée et les plans LSB.
5. Aller dans **Détecter**.
6. Tester d'abord l'image propre, puis le PNG stego.
7. Lire les probabilités SVM/RF et expliquer les seuils.
8. Aller dans **Extraire le texte**.
9. Charger le PNG stego et montrer la lecture de l'en-tête puis du payload.

26 Limites du système réalisé

- Le modèle est spécialisé dans le LSB généré par notre application.
- Les méthodes comme JMiPOD, J-UNIWARD et UERD demanderaient des données stego correspondantes.
- Les petits messages peuvent être plus difficiles à détecter si on retire l'indice d'en-tête propre à l'application.
- Les images JPEG recompressées peuvent détruire le message LSB.
- Les excellents scores ne doivent pas être présentés comme une stéganalyse universelle.

27 Ce qu'il faut retenir

Le projet relie directement le cours de traitement d'images à une application pratique. On part d'une matrice RGB, on manipule les bits, on visualise les canaux et les plans LSB, on extrait des caractéristiques de texture et de statistiques, puis on utilise un classificateur pour prendre une décision.